

Grundlagen der C-Programmierung

Dr. Henrik Brosenne
Georg-August Universität Göttingen
Institut für Informatik

Wintersemester 2024/25

Speicher auf dem Heap

- Speichertypen

- Funktionen zur Heapverwaltung

- Bereitstellung von Speicher

- Matrizen auf dem Heap

- Felder variabler Größe auf dem Heap

Speichertyp statischer Speicher

Statischer Speicher wird bereits vom Compiler bereitstellt, z.B. für Konstanten (und statische Variablen).

Der im statischen Speicher reservierte Speicherplatz steht während der gesamten Ausführung des Programms zur Verfügung.

Speichertyp Stack

(Automatische) Variablen, sind dynamisch.

Der erforderliche Speicher wird erst in dem Moment bereitgestellt, wenn bei der Ausführung des Programms der Block betreten wird, der ihre Definition enthält.

Der Speicher wird beim Verlassen des Blocks wieder freigegeben.

Diese Art von Variablen wird in der Regel auf einem **Stack** bereitgestellt.

Der Stack ist eine Datenstruktur, die linear wächst. D.h. Speicherplatz einer Variablen auf dem Stack kann erst wieder freigegeben werden, wenn alle später erzeugten Variablen ebenfalls wieder freigegeben wurden. Das entspricht genau der Verwendung von automatischen Variablen.

Der Stack ist ein **strukturierter dynamischer Speicher**.

Speichertyp Heap

Ein Programm hat die Möglichkeit, während es läuft Speicher vom Betriebssystem anzufordern, wobei sich die Größe des angeforderten Speichers erst bei Ablauf des Programms ergibt.

Dieser Speicher steht dem Programm dann so lange zur Verfügung, bis es ihn explizit wieder freigibt oder bis es terminiert.

Die Verfügbarkeit dieses Speichers ist insbesondere nicht an die Blockstruktur des Programms gebunden.

Man bezeichnet die Gesamtheit dieses Speichers als **Heap**.

Der Speicher auf dem Heap kann in beliebiger Reihenfolge angefordert und wieder freigegeben werden kann. Die Funktionen zur Anforderung von Speicher auf dem Heap müssen also einen gewissen Aufwand betreiben, um den Speicher zu verwalten.

Der Heap ist ein **unstrukturierter dynamischer Speicher**.

Speicherzuordnung

Bei statischem Speicher und bei Speicher auf dem Stack sorgt der Compiler dafür, dass eine Zuordnung zwischen Variablen und Adressen erfolgt.

Bei Speicher auf dem Heap erfolgt die Anforderung von Speicher vom Betriebssystem durch Aufruf einer entsprechenden Funktion.

Diese Funktion gibt einen Zeiger auf den Anfang des bereitgestellten Speichers als Funktionswert zurück, falls genug Speicher vorhanden ist.

Man ist **gezwungen**, diesen Wert in einer **passenden** Zeigervariablen zu speichern, damit man ihn später für die Zugriffe auf den Speicher verwenden kann.

Speicher auf dem Heap

Speichertypen

Funktionen zur Heapverwaltung

Bereitstellung von Speicher

Matrizen auf dem Heap

Felder variabler Größe auf dem Heap

Funktionen zur Heapverwaltung

C kennt Funktionen, mit denen man Speicher auf dem Heap anfordern kann, und eine Funktion zur Freigabe des Speichers.

Die Funktionen sind in der Standard-Headerdatei `stdlib.h` deklariert.

```
void *malloc(size_t groesse);  
void free(void *zeiger);
```

Die Funktion `malloc` stellt, wenn möglich, einen zusammenhängenden Speicherbereich der Größe `groesse` zur Verfügung und liefert als Funktionswert den Zeiger auf den Anfang dieses Speicherbereichs.

Die Größe bezieht sich dabei auf ein Vielfaches des Speicherbedarfs eines Zeichens.

Erinnerung. Der Standard schreibt `sizeof (char) == 1` vor.

Falls `malloc` den angeforderte Speicher nicht bereitstellen kann, ist der Funktionswert der Nullzeiger (`NULL`).

Die Speicherbereitstellung kann bei jedem Versuch fehlschlagen, deshalb sollte man den Rückgabewert von `malloc` **immer** auf `NULL` überprüfen.

malloc

`malloc` hat als Typ des Funktionswertes `void *`.

```
void *malloc(size_t groesse);
```

Ein Zeiger mit dem Typ `void *` kann nicht dereferenziert werden.

Ein `void`-Zeiger kann einer Variable jedes Zeigertyps zugewiesen werden, falls eine Typumwandlung nötig ist wird diese vom Compiler automatisch erledigt.

Der Rückgabewert von `malloc` kann aber auch mit einer explizit Typumwandlung (Castoperator) in den Typ der Zeigervariablen, der er zuweisen wird, umgewandelt werden, z.B. um Fehler auszuschließen oder die Lesbarkeit zu erhöhen.

Speicher, den man sich mit `malloc` beschafft, besitzt **keine** wohldefinierten Werte. Will man bestimmte Anfangswerte haben, muss man selber explizit für deren Zuweisung sorgen.

free

Die Funktion **free** gibt einen Speicherbereich wieder frei.

```
void free(void *zeiger);
```

Voraussetzung für ordnungsgemäßes Arbeiten der Funktion ist, dass man ihr nur Zeiger als Argument übergibt, die man sich zuvor mit **malloc** (oder einer anderen Bereitstellungsfunktionen) beschafft hat.

Ausnahme **NULL**. Wird **NULL** als Argument von **free** verwendet, tut die Funktion nichts.

Speicher auf dem Heap

Speichertypen

Funktionen zur Heapverwaltung

Bereitstellung von Speicher

Matrizen auf dem Heap

Felder variabler Größe auf dem Heap

Bereitstellung von Speicher (1/3)

Bei vielen mathematischen Programmen, die mit Vektoren und Matrizen arbeiten, ist in Regel erst zu Laufzeit bekannt, wie groß die Vektoren und Matrizen sind.

Dieses Problem kann man in C durch die Bereitstellung der benötigten Vektoren und Matrizen auf dem Heap lösen.

```
1  float *v;
2  int n;
3
4  // compute/read n
5
6  v = (float *) malloc(n * sizeof(float));
7
8  if (v == NULL) {
9      // error handling
10 }
11
12 // use v as vector
```

Zur Laufzeit wird gerade der Speicherplatz bereitgestellt, den ein `float`-Vektor mit `n` Komponenten benötigt.

Bereitstellung von Speicher (2/3)

Speicherplatz für eine $(n \times n)$ -Matrix mit `float`-Komponenten kann man sich genauso beschaffen.

```
1  float *m;
2  int n;
3
4  // compute/read n
5
6  m = (float *) malloc(n * n * sizeof(float));
7  if (m == NULL) {
8      // error handling
9  }
10
11 // use m as vector or cast to matrix
```

Bereitstellung von Speicher (3/3)

Zeigervariablen erlauben es immer, ihren Wert als Zeiger auf den Anfang eines Vektors zu interpretieren.

Ohne weiters ist es dagegen nicht möglich, Speicher auf dem Heap mit einer Matrixstruktur zu versehen. Man ist gezwungen die Speicherabbildungsfunktion selber zu definieren und sie für jeden Zugriff selber auszuwerten.

Der Zugriff mit eigener Speicherabbildungsfunktion funktioniert genau wie bei Felder, weil der in der Zeigervariablen gespeicherte Zeiger, als Zeiger auf den Anfang eines Vektors interpretiert werden kann.

Speicherfreigabe (1/2)

Speicher auf dem Heap steht dem Programm so lange zur Verfügung, bis es ihn wieder freigibt, aber um ihn nutzen zu können, braucht man zusätzlich auch einen Zeiger auf ihn.

Diese Zeiger können in der Regel selbst nicht auf dem Heap liegen, sondern müssen Variablen sein.

Formal ist folgende Funktion korrekt.

```
1 void produce_garbage(int n) {  
2     float *v;  
3     v = (float *) malloc(n * sizeof(float));  
4 }
```

Allerdings verschwindet ihre lokale Variable `v` mit Abschluss der Funktion wieder und mit ihr auch der Zeiger auf den Speicherbereich, den man sich mit `malloc` beschafft hat.

Mit einer Funktion wie `produce_garbage` kann man jedes Programm zum *Absturz* bringen, indem man sie oft genug mit einem großen Argument aufzuruft.

Bei jedem Aufruf wird Speicher angefordert. Da dieser Speicher nicht wieder freigegeben wird, muss der Heap notwendig irgendwann voll sein.

Speicherfreigabe (2/2)

Es ist also ein **schwerwiegender Programmierfehler**, Speicher auf dem Heap zu beschaffen, diesen aber nicht wieder freizugeben.

Wird der Speicher nur lokal in einer Funktion benötigt, so ist diese auch dafür zuständig, ihn wieder freizugeben.

```
1 void f(int n) {  
2     float *v;  
3     v = (float *) malloc(n * sizeof(float));  
4     // ...  
5     free (v);  
6 }
```

Speicherübergabe

Oft soll der in einer Funktion bereitgestellte Speicher in der rufenden Funktion weiterverwendet werden.

Dazu muss die rufende Funktion einen Zeiger auf den reservierten Speicherbereich erhalten, z.B. als Funktionswert.

Beispiel.

Kopieren eines String.

```
char *string_copy(const char *s)
```

Wenn die rufende Funktion einen Zeiger auf den reservierten Speicherbereich erhalten hat, muss diese dafür sorgen, dass der Speicherbereich wieder freigegeben wird.

Kopieren eines String (1/2)

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  char *string_copy(const char *s);
5
6  char *string_copy(const char *s) {
7      // string length
8      int n = 0;
9      while(s[n])
10         n++;
11
12     // allocate memory from the heap
13     char *t = malloc(n+1);
14     if (t == NULL)
15         return NULL;
16
17     // copy
18     n = 0;
19     while(t[n] = s[n]) {
20         n++;
21     }
22
23     return t;
24 }
```

Kopieren eines String (2/2)

```
26  int main (int argc, char *argv[]) {
27
28      char source[] = "Hello_World!";
29      char *target = string_copy(source);
30
31      printf("source:_%s\n", source);
32      printf("target:_%s\n", target);
33
34      free(target);
35
36      return 0;
37  }
```

Speicher auf dem Heap

Speichertypen

Funktionen zur Heapverwaltung

Bereitstellung von Speicher

Matrizen auf dem Heap

Felder variabler Größe auf dem Heap

Matrizen auf dem Heap

Ziel ist ein allgemeines Beispiel für beliebig dimensionale Matrizen mit vernünftige Indizierung.

Wie ist sind die Ausdrücke `m`, `m[i]`, `m[i][j]` für folgende Matrix zu interpretieren?

```
int m[2][3];
```

- Der Name `m` der Matrix repräsentiert einen (konstanten) Zeiger auf den Vektor der Zeiger auf die Zeilen.
- Der Ausdrucks `m[i]` ist ein (konstanter) Zeiger auf eine Zeile der Matrix.
- Paare eckiger Klammern sind Operatoren; aufeinanderfolgende Paare werden von links nach rechts abgearbeitet. Somit ist der Ausdruck `m[i][j]` ein Zeiger auf eine `int`-Speicherstelle.

Vektor von Vektoren

Es ist nicht festgeschrieben, dass bei doppelter Indizierung, etwa `m[i][j]`, der Teilausdruck `m[i]` eine Konstante sein **muss**, das kann man ausnutzen.

Aufgabe. Bereitstellung einer $n \times m$ -Matrix mit `double`-Komponenten.

Erste Schritt. Beschaffung eines Vektors mit n Komponenten, die jeweils den Zeiger auf eine Zeile aufnehmen können.

Zweiter Schritt. Beschaffung von Speicher für die Zeilen der Länge m und Eintragung der Zeiger auf die Zeilen in die Komponenten des im ersten Schritt beschafften Vektors.

```
1  double **a;
2  int n, m;
3  int i;
4
5  a = (double **) malloc(n * sizeof (double *));
6  if (a == NULL)
7      // error handling
8
9  for (i = 0; i < n; i++) {
10     a[i] = (double *) malloc(m * sizeof (double));
11     if (a[i] == NULL)
12         // error handling
13 }
```

Indizierung

Für den Ausdruck $a[i][j]$ wird zunächst $a[i]$ ausgewertet, das ist ein Zeiger auf die entsprechende Zeile. Danach erfolgt die zweite Indizierung und liefert den Wert der entsprechenden Komponente der Zeile.

Das Arbeiten mit Matrizen wird dadurch sehr einfach, da man so bereitgestellte Matrizen auch in Funktionen vernünftig indizieren kann.

Die Parameter, die die Matrizen repräsentieren, sind formal Vektoren (von Zeigern) und für Vektoren funktioniert die Indizierung auch in Funktionen.

Freigabe

Bei der Freigabe einer so bereitgestellten Matrix muss man offensichtlich genau umgekehrt vorgehen und erst die einzelnen Zeilen und dann den Vektor der Zeiger auf die Zeilen freigeben.

Es gibt noch etwas Optimierungspotenzial.

Die Funktionen `malloc` und `free` sind in der Regel ziemlich aufwendig, so dass man sie nicht unnötig oft aufrufen sollte.

Entsprechend ist es hier in der Regel günstiger, sich den gesamten Speicherbereich für die Matrix mit einem Aufruf von `malloc` zu beschaffen und diesen Speicher dann in einer Schleife aufzuteilen.

Beispiel (1/2)

```
1 double **matrixalloc(unsigned int n, unsigned int m) {
2     double **a;
3
4     a = malloc(n*sizeof(double *));
5     if (a != NULL) {
6         a[0] = malloc(n*m *sizeof(double));
7         if (a[0] == NULL) {
8             free(a);
9             a = NULL;
10        }
11        else {
12            unsigned int i;
13            for (i = 1; i < n; i++)
14                a[i] = a[i-1] + m;
15        }
16    }
17    return a;
18 }
```

Beispiel (2/2)

```
1 void matrixfree(double **a) {  
2     if (a != NULL)  
3         free(a[0]);  
4  
5     free(a);  
6 }
```

Speicher auf dem Heap

Speichertypen

Funktionen zur Heapverwaltung

Bereitstellung von Speicher

Matrizen auf dem Heap

Felder variabler Größe auf dem Heap

Beispiel

Funktionen zum Einlesen und Ausgeben einer Matrix bekannter Dimension.

```
1 void scan_matrix1(int n, int m, double matrix[n][m]) {
2     for(int i = 0; i < n; i++) {
3         for(int j = 0; j < m; j++) {
4             printf("matrix[%d][%d]: ", i, j);
5             scanf("%lf", &matrix[i][j]);
6         }
7     }
8 }
```

```
1 void print_matrix(int n, int m, double matrix[][m]) {
2     for(int i = 0; i < n; i++) {
3         for(int j = 0; j < m; j++) {
4             printf("%10.2lf", matrix[i][j]);
5         }
6         printf("\n");
7     }
8 }
```

Funktionsaufruf

```
1  int main() {  
2      int n, m;  
3  
4      printf("Dimension (nxm): ");  
5      scanf("%dx%d", &n, &m);  
6  
7      double a[n][m];  
8      scan_matrix1(n, m, a);  
9      ...  
10     print_matrix(n, m, a);  
11     ...  
12 }
```

Beispiel

Funktion zum Einlesen einer Matrix bekannter Dimension, wobei der Speicher für die Matrix von der Funktion bereitgestellt wird.

```
1  double* scan_matrix2(int n, int m) {
2      double (*matrix)[m];
3      matrix = malloc(n*m*sizeof(double));
4
5      for(int i = 0; i < n; i++) {
6          for(int j = 0; j < m; j++) {
7              printf("matrix[%d][%d]: ", i, j);
8              scanf("%lf", &matrix[i][j]);
9          }
10     }
11     return matrix[0];      // &matrix[0][0];
12 }
```

Beispiel

Funktionsaufruf

```
1  int main() {  
2      ...  
3      double (*b)[m];  
4      b = (double(*)[m]) scan_matrix2(n, m);  
5      ...  
6      print_matrix(n, m, b);  
7      ...  
8      free(b);  
9      ...  
10 }
```
